

OPERATING SYSTEMS AND SYSTEM PROGRAMMING

PRACTICAL – WEEK 9

Name: S Svara Haasini

Roll No: 2520090170

Task 1: Low-Level Linux File I/O vs Standard Library I/O

Question:

Write a program that compares the performance of low-level I/O (read, write, open, close) with standard library I/O functions (fread, fwrite, fopen, fclose). Measure and compare execution times.

Source Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <time.h>

#define BUFFER_SIZE 4096

double elapsed(struct timespec start, struct timespec end)
{
    return (end.tv_sec - start.tv_sec) +
           (end.tv_nsec - start.tv_nsec) / 1000000000.0;
}

int main(void)
{
    char buffer[BUFFER_SIZE];
    ssize_t n;
    struct timespec start, end;

    printf("=== Low-Level I/O vs Standard I/O ===\n\n");

    /* LOW-LEVEL I/O */
    int src = open("input.dat", O_RDONLY);
    int dst = open("low_copy.dat",
                  O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (src < 0 || dst < 0)
    {
        perror("open");
        return 1;
    }

    lseek(src, 0, SEEK_SET);

    clock_gettime(CLOCK_MONOTONIC, &start);

    while ((n = read(src, buffer, BUFFER_SIZE)) > 0)
        write(dst, buffer, n);

    clock_gettime(CLOCK_MONOTONIC, &end);
}
```

```
haasini@SSH: ~/OSSP_upload × + v
GNU nano 7.2 P9_T1.c *
while ((n = read(src, buffer, BUFFER_SIZE)) > 0)
    write(dst, buffer, n);

clock_gettime(CLOCK_MONOTONIC, &end);

close(src);
close(dst);

printf("Low-level I/O time : %.6f seconds\n",
      elapsed(start, end));

/* STANDARD LIBRARY I/O */
FILE *fin = fopen("input.dat", "rb");
FILE *fout = fopen("stdio_copy.dat", "wb");

if (fin == NULL || fout == NULL)
{
    perror("fopen");
    return 1;
}

clock_gettime(CLOCK_MONOTONIC, &start);

size_t bytes;

while ((bytes = fread(buffer, 1, BUFFER_SIZE, fin)) > 0)
    fwrite(buffer, 1, bytes, fout);

clock_gettime(CLOCK_MONOTONIC, &end);

fclose(fin);
fclose(fout);

printf("Standard I/O time : %.6f seconds\n",
      elapsed(start, end));

printf("\nCopies completed successfully.\n");

return 0;
}
```

Observations:

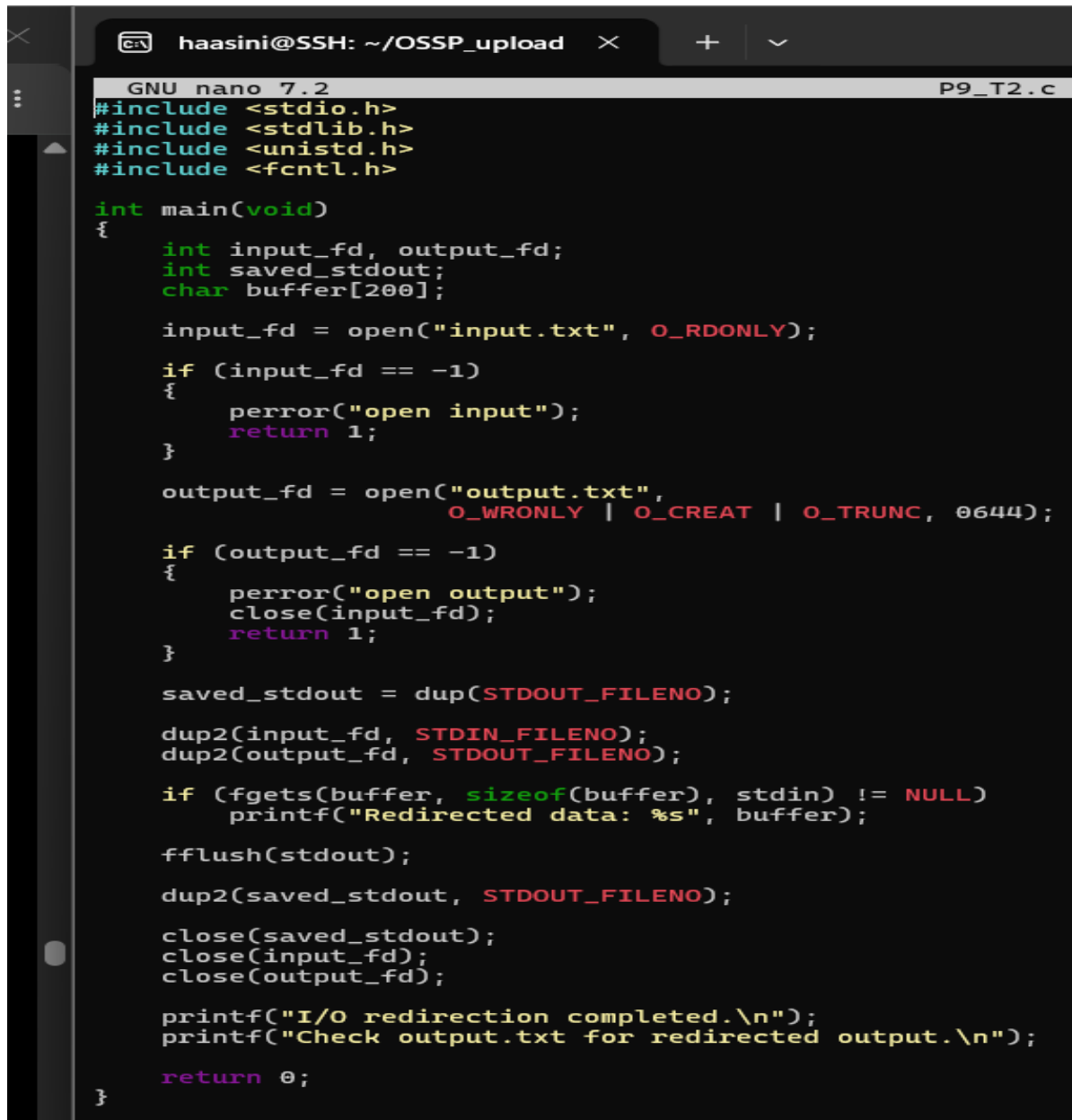
1. Low-level I/O functions (open, read, write, close) directly interact with the operating system.
2. Standard library I/O functions (fopen, fread, fwrite, fclose) provide buffered I/O with higher-level abstraction.
3. Low-level I/O uses file descriptors (integers) returned by open().
4. Standard I/O uses FILE pointers returned by fopen().
5. BUFFER_SIZE macro (4096 bytes) controls the chunk size for read/write operations.
6. clock_gettime() with CLOCK_MONOTONIC provides high-resolution timing measurement.
7. Both approaches copy file content with different I/O mechanisms and performance characteristics.
8. Standard I/O includes internal buffering which can improve performance for certain file sizes.
9. Low-level I/O provides more direct control but requires explicit buffer management.
10. Performance differences depend on buffer size and file size characteristics.

Task 2: Standard Input and Output Redirection Using dup2()

Question:

Develop a program that demonstrates I/O redirection using dup2(). Redirect stdin and stdout to files, demonstrating file descriptor manipulation and input/output redirection.

Source Code:



```
GNU nano 7.2 P9_T2.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>

int main(void)
{
    int input_fd, output_fd;
    int saved_stdout;
    char buffer[200];

    input_fd = open("input.txt", O_RDONLY);

    if (input_fd == -1)
    {
        perror("open input");
        return 1;
    }

    output_fd = open("output.txt",
                    O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (output_fd == -1)
    {
        perror("open output");
        close(input_fd);
        return 1;
    }

    saved_stdout = dup(STDOUT_FILENO);

    dup2(input_fd, STDIN_FILENO);
    dup2(output_fd, STDOUT_FILENO);

    if (fgets(buffer, sizeof(buffer), stdin) != NULL)
        printf("Redirected data: %s", buffer);

    fflush(stdout);

    dup2(saved_stdout, STDOUT_FILENO);

    close(saved_stdout);
    close(input_fd);
    close(output_fd);

    printf("I/O redirection completed.\n");
    printf("Check output.txt for redirected output.\n");

    return 0;
}
```

Output:

```
Copies completed successfully.
haasini@SSH:~/OSSP_upload/Practical_Week9$ nano P9_T2.c
haasini@SSH:~/OSSP_upload/Practical_Week9$ nano P9_T2.c
haasini@SSH:~/OSSP_upload/Practical_Week9$ echo "Hello from redirected standard input" > input.txt
haasini@SSH:~/OSSP_upload/Practical_Week9$ gcc -Wall -Wextra P9_T2.c -o P9_T2
./P9_T2
cat output.txt
I/O redirection completed.
Check output.txt for redirected output.
Redirected data: Hello from redirected standard input
haasini@SSH:~/OSSP_upload/Practical_Week9$ |
```

Observations:

1. dup2() duplicates a file descriptor to another file descriptor number.
2. dup2(input_fd, STDIN_FILENO) redirects standard input to read from a file.
3. dup2(output_fd, STDOUT_FILENO) redirects standard output to write to a file.
4. File descriptors allow redirection of standard I/O streams without modifying program logic.
5. The original file descriptors can be preserved using dup() before redirection.
6. Program output directed to output.txt instead of terminal using dup2 redirection.
7. Standard input can be redirected to read from input.txt using dup2.
8. fgets() and printf() automatically use redirected file descriptors.
9. File descriptor 0 is stdin, 1 is stdout, 2 is stderr.
10. dup2() enables flexible I/O redirection comparable to shell redirection operators.